Flood Monitor Spring Boot REST API

Sebastian Nosek 4AHIT

Projektziel

Das Ziel dieses Projekt war die Entwicklung einer REST basierten Spring Boot Anwendung zur Überwachung von Hochwassermessstationen und Messwerten. Mein Floodmonitor hat verschiedene Endpunkte, um Stationen nach Flüssen oder Warnstufen zu filtern und dann aufzurufen. Bei mir war das größte Problem, dass die Daten von den Endpunkten, im JSON als auch im XML-Format aufgerufen werden.

Messwerte simulieren

Angabe:

- Wasserstände dürfen nicht negativ sein.
- Der Akkustand liegt zwischen 0 und 100 Prozent.
- Starker Niederschlag soll den Wasserstand tendenziell erhöhen.
- Ein höherer Wasserstand soll zu einer höheren Durchflussmenge führen.
- Der Wasserstand soll sich zwischen zwei Messungen nicht völlig zufällig verändern.
- Bei sehr niedrigem Akkustand kann der Status auf `MAINTENANCE` wechseln.
- Für deaktivierte oder ausgefallene Stationen werden keine neuen Messungen erzeugt.

Meine Regeln:

1. Wenn eine Station Offline ist, werden keine neuen Daten mehr zurückgegeben, es wird auf UNKNOWN umgeschaltet.

```
if (!station.getIsActive() || station.getStatus() == Statio
nStatus.OFFLINE) {
```

```
                station.setWarningLevel(WarningLevel.UNKNOW
N);
                continue;
```

2. Die Batter sinkt max nur 2% pro Runde, also pro Refresh, aber sie wird niemals unter 0% sein

```
int neueBattery = Math.max(0, station.getBatteryLevel() - r
andom.nextInt(3));
            station.setBatteryLevel(neueBattery);
```

3. Wenn der Akku unter 20% ist, so wird die Station auf Maintenance gesetzt.

```
if (neueBattery < 20) {
                station.setStatus(StationStatus.MAINTENANC
E);
            }
```

4. Der Niederschlag ist nur zwischen 0.0 bis 5.0

```
double regen = runden(random.nextDouble() * 5.0);
            station.setRainfall(regen);
```

5. Der Wasserstand verändert sich hier nicht zuviel zwischen 2 Messungen, da es sonst unrealistisch wäre. Anbei wird auch starker Regen dazugerechnet, wenn es regnet, so steigt auch der Wasserstand um einiges.

```
double veraenderung = (random.nextDouble() * 4.0 - 1.8) +
(regen * 0.3);
            double neuerWasserstand = Math.max(0, station.g
etWaterLevel() + veraenderung);
            station.setWaterLevel(runden(neuerWasserstan
d));
```

6. Wenn die Wassermenge hoch ist, so muss auch die Durchflussmenge größer

```
station.setFlowRate(runden(station.getWaterLevel() * 2.2))
```

7. Die Temperatur darf sich nicht zuviel verändern, von dem vorherigen

```
station.setTemperature(runden(station.getTemperature() + (r
andom.nextDouble() - 0.5)));
```

8. Die Warnstufe darf laut der Angabe nicht vom Client kommen, so wird sie am Server berechnet

```
station.setWarningLevel(calculateWarninglevel(station));
```

CalculateWarninglevel

Unknowkn → Die Station ist Offline, es gibt also keine gültigen Messungen

Critcal → Wenn der Wasserstand niedriger oder gleich ist, als der kritische Wert.

Warning→ Wenn der Wasserstand niedriger oder gleich ist, als der Warnwert

Normal → alles andere

```
public WarningLevel calculateWarninglevel(Station station)
{
        if(!station.getIsActive() || station.getStatus()  =
= StationStatus.OFFLINE) {
            return WarningLevel.UNKNOWN;
        }
        if(station.getWaterLevel() >= station.getCriticalWa
terLevel()) {
            return WarningLevel.CRITICAL;
        }
        else if(station.getWaterLevel()>=station.getWarning
WaterLevel()) {
            return WarningLevel.WARNING;
        }
        else {
```

```
            return WarningLevel.NORMAL;
        }
    }
```

Content Netgotiation

Alle Endpunkte haben bei @GetMapping

```
produces={APPLICATION_JSON, APPLICATION_XML}
```

somit prüft Spring automatisch den AcceptHeader.

wichtig (war auch ein Problem bei mir)

Ein XML braucht immer genau ein Root-Element, weil die Endpunkte einfach nur Java Listen sind, dadurch gibt es zb kein <station> (außer ich hätte eine Wrapper Klasse gemacht), sondern XML benennt es automatisch auf <list> in meinem Fall.

Fehlerbehebungen

Die Fehlerbehebung findet im GlobalExceptionHandler statt, Fehler werden als Map mit timestamp, status, error, message und path gebaut, als JSON und XML

Statuscodes

200 → erfolgreiche Anfrage

400 → ungültige Daten

404 → not Found, also zb falsche ID

406 → nicht unterstütztes Format

500 → Serverfehler

automatisierte Tests

Die Aufgabe bei den JUnit Tests war es, dass sie nie aus dem nix fehlschlagen dürfen, bzw der Build immer successfull sein soll, dass habe ich mit Mocking gemacht, damit kann man Klassen sozusagen manipulieren, aber nur für die JUnit Tests, damit sie dann mit den manipulierten Klassen getestet werden können und somit nicht aus dem nichts fehlschlagen.

Beispiel

```
@Test
    public void test1_GetStationsJson() throws Exception {
        when(simulationService.getfilterStationen(any(), an
y(), any())) //any(), brauche ich, wenn der Filter nach etw
as fragt, damit der Wert egal ist.
                .thenReturn(Collections.singletonList(testS
tation("1")));       //es antwortet mit einer singletonList
e, das heißt eine Liste mit nur einem Eintrag

        mockMvc.perform(get("/api/v1/stations").accept(Medi
aType.APPLICATION_JSON))     //hier wird mit der GET api/v
1/stations gemockt
                .andExpect(status().isOk())
                .andExpect(content().contentType(MediaType.
APPLICATION_JSON))
                .andExpect(jsonPath("$[0].id").value("1"));
    }
```

Endpunkte

Beispiel:

```
@GetMapping(value = "/stations", produces = {MediaType.APPL
ICATION_JSON_VALUE, MediaType.APPLICATION_XML_VALUE})
    public List<Station> getStations(@RequestParam(required
= false) String river,@RequestParam(required = false) Boole
an isActive, @RequestParam(required = false) WarningLevel w
arningLevel) {
        return simulationService.getfilterStationen(river,
isActive, warningLevel);
    }
```

Problem

Format: JSON Stationen laden

1) Messstationen

ID	Name	Fluss	Zeitpunkt	Wasserstand (cm)	Durchfluss (m³/s)	Niederschlag	Akku (%)	Status	Warnstufe
1	Wien Donauinsel	Donau	2026-10-11T13:24:17.0261025	257.5	566.5	1.9	89	ONLINE	NORMAL
2	Linz Donau	Donau	2026-10-11T13:24:17.0261025	343.5	755.7	1.6	88	ONLINE	WARNING
3	Köln Rhein	Rhein	2026-10-11T13:24:17.0261025	534.1	1175	3.3	94	ONLINE	CRITICAL
4	Krakow	Wisla	2026-10-11T13:20:41.9142123	260	572	0	100	ONLINE	UNKNOWN

Flood Monitor

Daten geladen um 13:21:16

Format: XML Stationen laden

1) Messstationen

ID	Name	Fluss	Zeitpunkt	Wasserstand (cm)	Durchfluss (m³/s)	Niederschlag	Akku (%)	Status	Warnstufe

2) Aktuellster Messwert

Mein größtes Problem war es, die XML Daten anzeigen zu lassen, als JSON Format werden mir die ganzen Daten angezeigt, doch als XML Format wird es mir nur durch den Abruf vom Link angezeigt, zb wenn ich http://localhost:8080/api/v1/alerts eingebe, bekomme ich ein XML format:

This XML file does not appear to have any style information associated with it. The document tree is shown below.

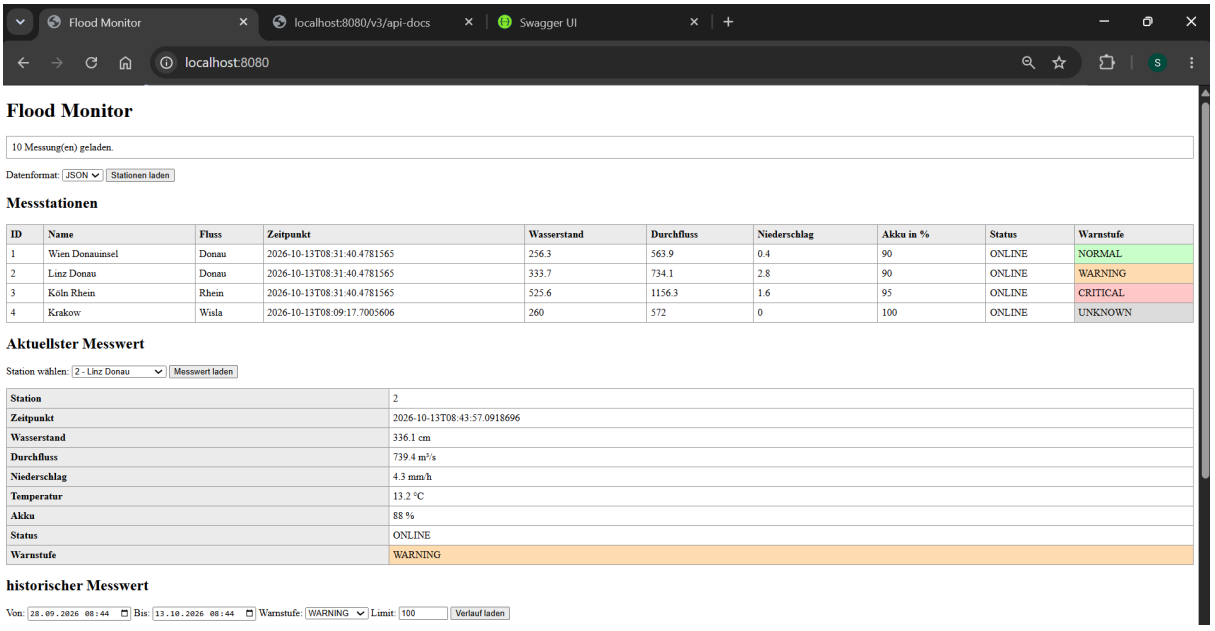
```
▼<List>
  ▼<item>
     <stationId>2</stationId>
     <timestamp>2026-10-11T13:25:27.2173122</timestamp>
     <waterLevel>348.0</waterLevel>
     <flowRate>765.6</flowRate>
     <rainfall>4.7</rainfall>
     <temperature>11.6</temperature>
     <batteryLevel>82</batteryLevel>
     <status>ONLINE</status>
     <warningLevel>WARNING</warningLevel>
   </item>
  ▼<item>
     <stationId>3</stationId>
     <timestamp>2026-10-11T13:25:27.2173122</timestamp>
     <waterLevel>543.5</waterLevel>
     <flowRate>1195.7</flowRate>
     <rainfall>2.3</rainfall>
     <temperature>11.4</temperature>
     <batteryLevel>88</batteryLevel>
     <status>ONLINE</status>
     <warningLevel>CRITICAL</warningLevel>
   </item>
 </List>
```

Mir war aber bewusst, dass der Fehler in der index.html liegen muss, da die Daten ja gespeichert und geladen werden, trotzdem war ich mir da unsicher und hab die KI gefragt mit dem prompt: Ich habe ein Problem, ich muss bei einer Tabelle insgesamt 2 Formate zurückbekommen, einmal im JSON Format und einmal als XML, doch wenn ich XML laden will, wird die Tabelle leer und anbei hab ich ihm die Index.html geschickt. Der Fehler war, meine Methode hat nach ‘station’ gesucht, obwohl das XML sich selber einen Art Umschlag benannt, bei mir mit <List> und <item>, deswegen hat die Methode nie etwas mit <station> gefunden und somit war die Tabelle auch leer.

Behoben habe ich es dann mit: $(daten).find('item'), damit es direkt auf das item zugreifen kann, welches auch in der XML existiert.

Bei der Filterung vom River in der Methode getfilterStationen, hatte ich bei der Filterung mit river ein ! vergessen, wodurch der Filter dann eigentlich richtige Flüsse nicht erlaubt, gefunden habe ich es als ich in der url: http://localhost:8080/api/v1/stations?river=Donau eingegeben habe.

Web-Oberfläche



Fragestellung

- Was sind die Hauptmerkmale von JSON?
  - leicht verständlicher Text, der aus Value Paaren und Arrays besteht, wie in einer Map (Schlüssen und Wert)
  - wurde entwickelt um Daten zwischen einem Server und einer Webanwendung zu übertragen
- Was sind die Hauptmerkmale von XML?
  - XML ist ein textbasiertes Format, welches Tags verwendet um die Elemente innerhalb eines Dokumentes zu definieren
  - hohe Flexibilität bei der Strukturierung von Daten
  - Jedes Element kann auch Attribute erhalten

- Wie unterscheidet die Applikation in diesem Beispiel, ob die Rückgabewerte in JSON oder XML zurückgegeben werden?
    - Durch den Content Negotiation, in meinem Code ist es produces = {}, im AcceptHeader kann ich entscheiden, ob ich XML oder JSON will und dadurch erkennt mein Programm jetzt, dass diese 2 Formate erlaubt sind und durch die Jackson XML implementierung kann ich JSON als auch ein XML bekommen.

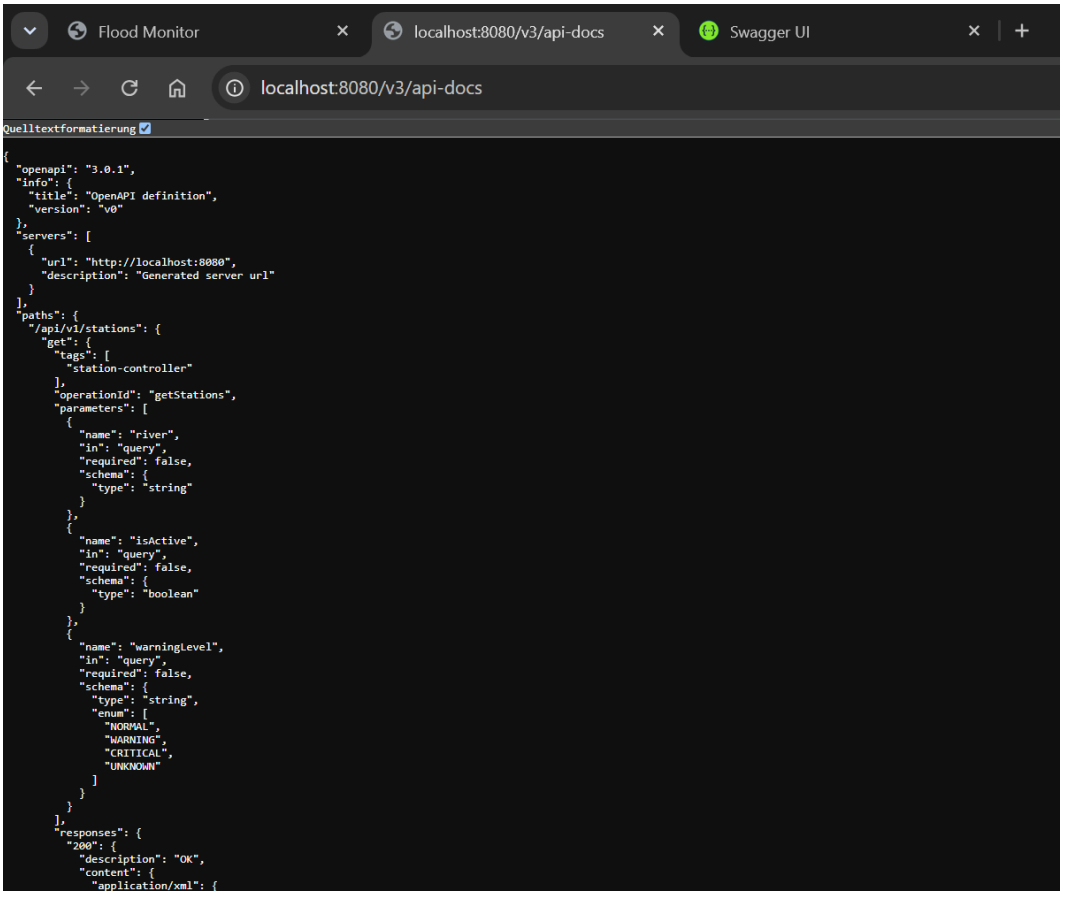
- Was zeichnet ein REST Interface aus und warum wird es heute sehr oft als standardisierte Schnittstelle verwendet?
    - Ein Rest Interface stellt Webdienste über ganz normale HTTP Anfragen bereit, über definierte Endpunkte und Methoden wie GET und POST können Clients Ressourcen auf dem Server anlegen oder abrufen.

- Was ist der Unterschied zwischen den HTTP Methoden GET, POST, PUT und PATCH?
    - GET → Daten abrufen
    - POST → Daten erstellen
    - PUT → Daten aktualisieren
    - PATCH → Teile von Daten aktualisieren

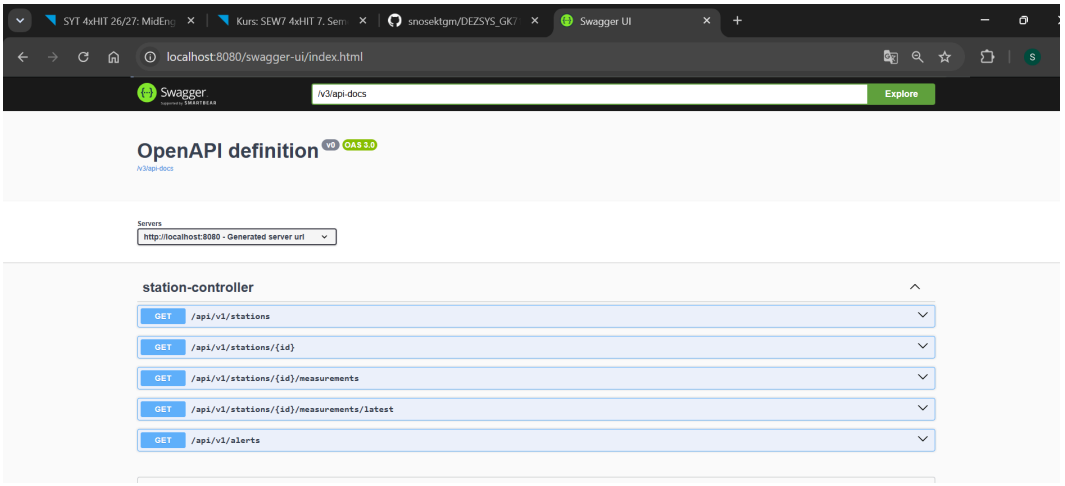
- Ich will eine neue Request Methode / Pfad in meinem REST Interface zur Verfügung stellen, welche Stellen des Quellcodes muss ich verändern?
    - Dafür muss ich das RequestMapping(””) vor dem Klassennamen verändern, dass ist ein Startpfad, so weiß jede Mapping methode, dass sie immer mit dem Pfad beginnen muss.

- Kann man OpenAPI und Swagger vergleichen? Wenn ja, dann vergleichen Sie beide Themengebiete. Wenn nein, dann begründe die Antwort.
    - Ja man kann die beiden vergleichen, dass sind die Unterschiede
    - Open API

- Open API ist ein Standard zur maschinenlesbaren Beschreibung von REST APIs in JSON oder YAML. Es ermöglicht die einfache Erstellung interaktiver sowie statischer Dokumentationen und unterstützt das automatische Generieren von Testfällen und Mock-Servern.



- Swagger UI
  - Swagger UI ist eine Art Test Web-Interface, damit ich zb schauen kann ob meine Mapping Methoden bzw meine Endpunkte so funktionieren, wie sie funktionieren sollten, die Oberfläche sieht so aus:



- Wo liegt im folgenden Code-Snippet der Fehler? Begründe die Antwort.

```
@GetMapping("/stations/add")
public String createStation(@RequestParam String name, @Req
uestParam double warning) {
    stationService.save(new Station(name, warning));
    return "Station successfully created!";
}
```

In diesem Beispiel wird GetMapping benutzt, dass heißt man kann sich Daten anzeigen lassen, doch die Methode darin (createStation), erstellt von sich aus eine neue Station und fügt sie dann mit add hinzu. Das funktioniert nicht, da GET nur Daten abrufen lässt, wenn man etwas erstellen will dann benutzt man Post.

Verbessert würde es so aussehen: @PostMapping(”/stations/add”)

Quellenverweis Fragestellung

1. Wissen über die Endpunkte → Web-Endpunkte.pdf aus dem SEW Kurs
2. OpenAPI VS Swagger UI → https://lab.wallarm.com/what/openapi-vs-swagger-eine-ausfuehrliche-klaerung/?lang=de
3. REST-Interface → https://www.knguru.de/blog/was-ist-eine-rest-api
4. JSON - XML → https://contabo.com/blog/de/xml-vs-json/

KI-Prompts

Index.html → “Kannst du mir bitte helfen eine Index.html für mein Projekt zu erstellen, es geht um Messtation am Fluss, die Daten wie den Wasserstand, Akku oder Warninglevel messen und diese dann entsprechend zurückgeben. Mein Code funktioniert schon, wenn ich es in der URL abrufe, doch jetzt brauche ich eine Index.html, die soll folgendermaßen aussehen: Drei Tabellen, die erste Tabelle ist, wo alle Messtationen angezeigt werden, die zweite Tabelle soll nur den neusten Messwert einer Messtation anzeigen und die letzte Tabelle soll den historischen Messwert einer Messtation anzeigen, wo man von - bis auswählen kann und das Warninglevel selbst, also Normal, Warning, Critical, wichtig ist, dass man aussuchen kann, ob man die Daten als JSON oder XML in der Tabelle haben will. (Anbei habe ich dann meinen Code gesendet)”

Test Klasse → “Kannst du mir anhand meines Codes zeigen, wie Mockito funktioniert, also das manipulieren von Klassen, damit die JUnit Tests nicht aus dem nix fehlschlagen, zeig mir bitte wie ich das machen kann, aber auch wie ich von selbst aus Exceptions werfen kann und erkläre mir es genau.”

Simulation → Die KI hat mir eine Simulation mit @Schedulde erstellt, doch ich habe es nicht wirklich verstanden, es war bisschen kompliziert, dass pro bei Schedulded ist aber, dass es die Werte automatisch in einem Takt oder Intervall zurückgibt ohne, dass man den Button drücken muss. Trotzdem wollte ich die Logik dahinter verstehen also habe ich gesagt, dass ich es ohne @Schedule machen will: “Mach es bitte ohne @Scheduled und erkläre mir wie du das gemacht hast, damit ich es verstehe”

Probleme → “Warum werden in meiner Tabelle die XML-Daten nicht angezeigt? ”, “Wieso werden die Daten bei einer URL-Anfrage direkt in XML zurückgegeben und nicht in JSON?”, “Kannst du dir meinen GlobalExceptionHandler anschauen und es verbessern, damit es laut der Angabe richtig ist, mach es bitte ohne Wrapper Klassen, erkläre mir aber davor was du machen willst. ”